Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila. 2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

- ¿Qué estructura de datos representa la fila de tarjetas? Un arreglo unidimensional.
- ¿Qué ciclo representa las **pasadas** y cuál representa las **comparaciones** dentro de una pasada? Puede ser un ciclo mientras para las pasadas y un for para poder hacer las comparaciones.
- ¿Cómo se realiza el **intercambio** de dos elementos sin perder uno de ellos? Mediante una variable auxiliar. ¿Cómo sabe el algoritmo que **ya terminó**, es decir, cuándo detenerse antes de completar todas las pasadas?Con una variable booleana que inicia en falso al comienzo de cada pasada. Si hace una pasada completa sin ningún intercambio, significa que el arreglo ya está ordenado y el ciclo finaliza.
- ¿Cuántos elementos hay que comparar en cada pasada? Piensen en qué tarjetas ya quedaron en su lugar definitivo. Por cada pasada se utiliza la regla “n-1” hasta llegar al indice [0,0]

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

```
Algoritmo OrdenamientoBurbuja

    Definir n, i, j, aux Como Entero

    Definir totalComparaciones, totalIntercambios, totalPasadas Como Entero
```

```
Definir huboIntercambio Como Logico



n <- 5

Dimension arreglo[5]



// Asignación de valores usando índices del 1 al 5

arreglo[1] <- 34

arreglo[2] <- 12

arreglo[3] <- 25

arreglo[4] <- 8

arreglo[5] <- 19



Escribir "Arreglo Inicial: [34, 12, 25, 8, 19]"

Escribir "----------------------------------------"



totalComparaciones <- 0

totalIntercambios <- 0
```

```
totalPasadas <- 0

i <- 1

huboIntercambio <- Verdadero



// Ciclo principal de pasadas

Mientras (i <= n - 1) Y (huboIntercambio == Verdadero) Hacer

    huboIntercambio <- Falso

    totalPasadas <- totalPasadas + 1



    // Ciclo interno de comparaciones adyacentes (del 1 hasta n - i)

    Para j <- 1 Hasta (n - i) Con Paso 1 Hacer

        totalComparaciones <- totalComparaciones + 1



        Si arreglo[j] > arreglo[j + 1] Entonces

            // Intercambio utilizando variable auxiliar

            aux <- arreglo[j]

            arreglo[j] <- arreglo[j + 1]
```

```
            arreglo[j + 1] <- aux



            totalIntercambios <- totalIntercambios + 1

            huboIntercambio <- Verdadero

        FinSi

    FinPara



    // Imprimir estado de la fila al finalizar cada pasada

    Escribir "Pasada ", totalPasadas, ": [", arreglo[1], ", ", arreglo[2], ", ", arreglo[3], ", ",
arreglo[4], ", ", arreglo[5], "]"



    i <- i + 1

FinMientras



Escribir "----------------------------------------"

Escribir "Resumen de Ejecución:"

Escribir "Total Pasadas: ", totalPasadas
```

```
    Escribir "Total Comparaciones: ", totalComparaciones

    Escribir "Total Intercambios: ", totalIntercambios



FinAlgoritmo
```

Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo). 2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**. 4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso. 5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

Fase 4: Reflexión

- ¿Coincide la salida del programa con lo que registraron con las tarjetas? Sí, la salida del programa en Java coincide de forma exacta con los estados intermedios y finales que se registraron durante la simulación física con las tarjetas. Tanto los contadores de pasadas como los de comparaciones e intercambios son congruentes.
- ¿Qué cambios tuvo que hacer su pseudocódigo al pasarlo a Java? Se estructuró el código bajo la sintaxis formal de clases y métodos de Java, también se ajustó el manejo de los índices del arreglo para que comiencen en 0 (tal como lo requiere el lenguaje).
- ¿Qué pasa cuando el arreglo ya está ordenado? ¿Sigue comparando? Gracias a la implementación de la bandera de control huboIntercambio, el algoritmo realiza únicamente una pasada inicial de verificación. Al notar que no se produjo ningún intercambio, el ciclo se interrumpe de inmediato, evitando pasadas y comparaciones redundantes.
- ¿Es eficiente con listas muy grandes? Relacionen su respuesta con el número de comparaciones ($O(n^2)$). No es eficiente al trabajar con listas o arreglos de gran tamaño. Al tener una complejidad temporal , el número de operaciones de comparación e intercambio crece de forma cuadrática conforme aumenta el número de elementos (n), lo que satura el rendimiento en comparación con algoritmos más avanzados.

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e			

intercambios por pasada).

2 **Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.

3 **Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).

4 **Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.

5 **Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.